PROOFBOARD

CLI Engineer Specification

Version 1.8 | June 2026

Implementation language: Go | Open source: MIT | Audience: CLI Engineering team

> **LEGAL NOTE**
> This revision makes two structural changes from v1.6: (1) the remote org handshake (Phase 6) is removed entirely and replaced with local-only fraud detection, which also extends the product to engineers without current repository access; (2) the Proof Card's organisation name field is decoupled from the classification pipeline and becomes a free-text, engineer-authored field — the same category of disclosure as a CV entry — clearly separated from the algorithm-derived data around it. This document is an engineering specification, not legal advice. Proofboard should obtain review from a qualified employment / IP / data-privacy attorney before any feature in this document is used against a third party's private repository at scale.

Summary of changes from v1.6:

- Phase 6 (Org Handshake) is REMOVED. No live network connection to any remote repository host occurs at any point in the pipeline.
- Local-only fraud detection introduced to replace the handshake's integrity function: GPG/SSH commit signature verification (where present), temporal rhythm / entropy analysis, and existing aiNoiseScore boilerplate detection — all running locally, all unchanged in terms of data transmitted.
- Engineers without current repository access (former employees with a retained local clone) can now generate a proof, labelled distinctly on the Proof Card. This was previously blocked by the handshake requirement.
- Proof Card organisation/company name field is now engineer-authored free text, decoupled from the CLI's salted org hash and from the classification pipeline. Labelled on the card as self-reported.
- All four mitigations from v1.6 — no Wasm upload, generic category vocabulary, salted hashing, DNS kill switch — remain unchanged and in effect.

1. Overview

The Proofboard CLI is a local-first Go binary that runs on the engineer's machine. It reads Git commit history from the local .git directory, classifies the work into abstract technical categories, destroys all proprietary text and identifying strings before transmission, and sends only a clean, salted cryptographic footprint to the Proofboard API. No live network connection to any third-party repository host occurs at any point in this process.

The CLI is the product. OAuth is the fallback path for engineers who cannot or choose not to install it.

Property	Value
Language	Go
Licence	MIT — open source from Day 1
Binary type	Statically compiled — zero runtime dependencies
Minimum Go version	1.21
CLI entry point	proofboard
Core commands	auth · link · sync · status · logs
Sync triggers	Post-merge hook (primary) + post-pull hook (secondary). No persistent background daemon.
Network access to repo host	NONE. No handshake, no live connection of any kind to the repository's remote. All verification is local-only — see Section 6.
API communication	HTTPS only, to Proofboard's own servers — JWT-signed payloads. No communication with the repository's host (GitHub, GitLab, etc.) of any kind.
Data destroyed locally	Commit message text, file paths, raw repository name, raw organisation name, author email (pre-hashing)
Data transmitted to Proofboard	Salted SHA256 hashes, timestamps, numerical stats, abstract technical category labels, salted org hash, salted email hash, local fraud-signal scores

2. Binary Distribution

Unchanged from v1.6. The CLI ships as pre-compiled static binaries. No package manager dependency required. No Node.js. Direct download or install script.

2.1 Build Targets

Platform	Architecture	Binary Name
macOS	arm64 (Apple Silicon)	proofboard-darwin-arm64
macOS	x86_64 (Intel)	proofboard-darwin-amd64
Linux	x86_64	proofboard-linux-amd64
Windows	x86_64	proofboard-windows-amd64.exe

2.2 Code Signing

Unchanged. macOS notarization and Windows Authenticode signing are required before public launch.

3. Commands

CHANGED IN v1.7

proofboard link no longer performs a live handshake against the repository's remote. It now only reads local Git configuration (the remote URL string, read locally from .git/config, never connected to) to derive the salted org/repo hash, and checks the DNS kill switch (Section 5C). No outbound connection to the repository host occurs.

Command	Behaviour
proofboard auth	Opens the engineer's default browser to https://app.proofboard.io/cli-auth. CLI listens on local port 9876 for the OAuth callback, stores the JWT in ~/.proofboard/credentials.json. Issues perEngineerSalt (Section 4A).
proofboard link	Must be run inside a Git repository directory, current or historical. Reads the remote URL string from local Git config only — no connection attempted. Derives the salted org hash and salted repo hash. Checks the DNS kill switch for the domain found in the remote URL string. Registers the repo against the authenticated account.
proofboard sync	Runs the full classification pipeline for the linked repository, including local fraud-detection scoring (Section 6), and transmits the clean salted payload.
proofboard status	Prints the current sync state for all linked repositories, including whether each was captured with current or no-longer-current repository access (Section 6.4).
proofboard logs	Prints the last 100 lines of ~/.proofboard/sync.log.
proofboard update	Downloads and replaces the binary with the latest release.
proofboard unlink	Removes the post-merge and post-rewrite hooks from .git/hooks/ and clears the repository entry from local state.
proofboard killswitch check {domain}	Checks whether a domain has published a Proofboard opt-out DNS TXT record.

REMOVED

proofboard sync --skip-handshake and proofboard sync --skip-handshake fallback documentation from v1.6/v1.4 are removed. There is no handshake to skip — this is now the only mode of operation.

4. The Classification Pipeline

> **CHANGED IN v1.7**
> The pipeline is now six phases, not eight. Phase 6 (Org Handshake) is removed. What was Phases 7 and 8 in prior versions are now Phases 6 and 7. Local fraud-detection scoring, previously a passive anti-fraud signal layer running alongside the handshake, is now the primary integrity mechanism and is detailed in its own section (Section 6) rather than folded into payload assembly.

The pipeline runs entirely on the developer's local machine. All phases run before any network communication to Proofboard's API. There is no network communication to the repository's remote host at any point.

Phase 1 — Local Ingest

Unchanged in mechanism. Strict in-memory constraint: commit text must never touch disk outside .git at any point in the pipeline.

```
git log --format="%H|%ae|%at|%s" --numstat --no-merges --author=$(git config user.email)
```

This command works identically whether the repository's remote is currently reachable or not, since it reads only the local .git object database. This is what allows Phase 1 onward to function for engineers without current repository access — see Section 6.4.

Phase 2 — Classification Engine

Two-layer hybrid classification, both layers local. No commit text transmitted at any stage. Mechanically unchanged from v1.6.

Category Dictionary

Unchanged from v1.6. Abstract technical vocabulary only — see table below for reference.

Category Label (current)	Path Signals (sample)
Identity & Access Layer	auth/, middleware/, guards/, security/
Presentation Layer	components/, pages/, .tsx, .jsx, .vue
Service & Interface Layer	api/, services/, controllers/, routes/
Data Persistence Layer	migrations/, models/, db/, repositories/
Infrastructure & Deployment Layer	infra/, .github/, Dockerfile, .yml
Performance & Caching Layer	next.config, vite.config, webpack/
Transaction Processing Systems	payments/, billing/, subscriptions/
Verification & Testing Layer	__tests__/, .test., .spec., e2e/

Phase 3 — Scoring Matrix

Unchanged. Top four categories by score become the project's Contribution Areas.

Phase 4 — Milestone Cluster Detection

Unchanged mechanically. clusterLabel uses the abstract vocabulary above.

Phase 5 — The Shredder

Unchanged from v1.6. Commit subject lines, file paths, author email, and repository/org names are all destroyed locally before any network call. Org/repo/email identifiers are hashed using the per-engineer salt described in Section 4A.

4A. Salted Identity Hashing

Unchanged from v1.6. Organisation and repository identifiers are never transmitted as static, guessable hashes. The CLI combines them with a per-engineer, server-issued salt before hashing, so the same company hashed by two engineers produces two different values and cannot be reverse-matched by an outside party running a guess-and-check attack against known company names.

Step	Detail
1	On first proofboard auth, the server issues a unique, random 256-bit perEngineerSalt, stored locally in ~/.proofboard/credentials.json (0600 permissions).
2	On proofboard link, the CLI computes orgHash = SHA256(perEngineerSalt + ':' + provider + ':' + orgName), derived entirely from the locally-read remote URL string. No connection to that remote is made to derive this.
3	These salted hashes are what is included in the payload — never the raw org/repo name, never the salt itself.

5. Corporate Opt-Out / Kill Switch

Unchanged from v1.6 in purpose. One mechanical update: the domain check is now performed against the remote URL string read locally from .git/config — since the CLI no longer connects to the remote at all, the kill-switch DNS lookup is the only outbound network reference to anything related to the repository's host, and it queries public DNS infrastructure only, never the repository host itself.

Result	CLI Behaviour
No TXT record found	Default — proceed normally. Absence of a record is not treated as consent, only as no objection.
TXT record = proofboard-allowed=false	Hard abort at proofboard link. No data is read, classified, or transmitted for that repository.
TXT record = proofboard-allowed=true	Explicit opt-in. Proceeds normally.

Published at proofboard.io/opt-out with copy-paste DNS configuration instructions, unchanged from v1.6.

6. Local Fraud Detection (Replaces the Org Handshake)

> **LEGAL NOTE**
> Removing the live network handshake to the repository's remote (formerly Phase 6) eliminates the one point in the pipeline where the CLI made an authenticated outbound connection to infrastructure the engineer does not own. It also means Proofboard can no longer cryptographically confirm, at the moment of capture, that the engineer currently has live push/pull access to the repository in question. This section documents the local-only mechanisms that now carry the fraud-prevention burden the handshake previously shared.

6.1 GPG / SSH Commit Signature Verification

If commits in the local repository are cryptographically signed — common in security-conscious engineering organisations, though not universal — the CLI extracts the signature block during Phase 1 ingest and verifies it locally against the corresponding public key.

Property	Spec
What it proves	That the signing key — not the company, not the repository — produced this specific commit. Forging a valid signature without the private key is computationally infeasible.
What it does NOT prove	That the engineer currently has push/pull access to the remote. Signature verification is a statement about the past act of committing, not present-day access.
Coverage	Partial. Most engineers do not sign commits by default. This is treated as a trust bonus signal when present, not a required gate.
Payload field	commitSignatureVerified: boolean. signedCommitRatio: float (proportion of commits in the synced range that carry a valid signature).

6.2 Temporal Rhythm and Entropy Analysis

Extends the existing aiNoiseScore logic from prior versions. Fabricated commit histories generated by scripts tend to show unnaturally uniform timing distributions; real human activity is irregular by nature.

Signal	What It Checks
commitIntervalVariance	Statistical variance in time gaps between consecutive commits. Near-zero variance across a large commit set is a red flag.
timeOfDayDistribution	Whether commit timestamps cluster in plausible human working-hour patterns versus a flat or perfectly random distribution.
burstPatternScore	Detects implausible bursts (e.g. thousands of commits generated in a tight script-driven loop with inhuman regularity).

This is a probabilistic signal, not a binary gate. It contributes to the existing aiNoiseScore and boilerplateSignal already defined in the anti-fraud table (Section 9) and is treated the same way — a scoring input, not an automatic rejection.

6.3 Pre-Existing Anti-Fraud Signals (Unchanged)

singleCommitRepoCap, boilerplateSignal, and identityMismatch from prior versions are unchanged in mechanism and continue to function identically, since they were never dependent on the handshake.

6.4 No-Current-Access Capture — New Capability

CHANGED IN v1.7

Because Phase 1 (local git log ingest) only ever required a local .git directory and never required live remote access, removing the handshake requirement means engineers who no longer have current access to a repository's remote — for example, a former employee with a retained local clone — can now generate a proof from that local history. This was previously blocked specifically because the handshake would fail for anyone without current access.

Field	Spec
accessStatus	New payload field. Values: 'current' (handshake would have succeeded, had one been attempted) is no longer determinable and is not claimed. The CLI does not assert current access status at all — see below.
Proof Card label	Every proof generated under this architecture is labelled identically: 'Locally Verified.' The card does not distinguish between current and former employment, because the CLI has no way to determine this and does not claim to.
What is NOT claimed	The CLI never asserts, implies, or labels a proof as confirming current repository access. No claim of present-day authorization is made for any proof, current employee or former.

This is a deliberate product expansion, not an incidental side effect to be hidden. Engineers who have left a company, including companies that are no longer operating, and engineers currently employed, both produce a Locally Verified proof through the identical mechanism. The CLI's local fraud-detection layer (6.1–6.3) is what stands in for the verification the handshake previously provided, applied identically regardless of the engineer's current employment status.

7. Anti-Fraud Signal Reference

Signal	How It's Captured	Server Action
orgHashMismatch	Salted org hash from locally-read remote URL string does not match the salted org hash stored at link time.	-40 to Internal Trust Score
identityMismatch	Salted author email hash on a commit does not match the authenticated account's salted email hash.	-30 per mismatch event
singleCommitRepoCap	Commit count below 5 at sync time.	Score capped regardless of other signals
boilerplateSignal / aiNoiseScore	Layer 1 classification noise score, now incorporating temporal entropy analysis (Section 6.2). Values above 0.8 indicate likely fabrication.	High score triggers -20 penalty and manual review flag
commitSignatureVerified	GPG/SSH signature present and valid (Section 6.1).	+10 trust bonus when present and valid. No penalty when absent — most engineers don't sign commits.

8. Proof Card — Disclosure Architecture

> **LEGAL NOTE**
> This section documents a structural change to how the Proof Card represents employer identity, distinct from anything in the CLI pipeline itself. It governs the web application's rendering of proof data, not the CLI, but is included here because it directly affects what the CLI's classification output is permitted to be paired with downstream.

8.1 The Problem This Section Resolves

Earlier proof card designs rendered the organisation name as if it were part of the same algorithmically-derived dataset as the work-category percentages and commit counts — all presented in one visual block with no distinction in provenance. This created a card that read as 'the system says this named company had this person doing this percentage breakdown of work,' which is a meaningfully more specific and more system-asserted claim than what the underlying pipeline actually verifies.

8.2 The Fix — Decoupling Disclosure from Verification

Layer	Source	Nature of the Claim
Organisation / company name	Free text, typed directly by the engineer. Never pulled from the salted org hash, never autocompleted or suggested from any CLI-derived data.	Engineer's own voluntary disclosure — the same category of statement as a CV entry or LinkedIn job listing.
Role, dates, duration	Engineer-entered, same as above.	Engineer's own disclosure.
Commit counts, PR counts, work category percentages, milestone clusters	CLI-derived, algorithm-classified, locally processed before transmission.	System-verified activity data. Says nothing about a named employer specifically — it is data about the engineer's verified work pattern.
Milestone outcome statements	Engineer-authored, editable text describing what was achieved.	Engineer's own narrative, same as a CV bullet point.

8.3 Card Labelling Requirement

The Proof Card must visually and textually indicate that the organisation name is self-reported. Required label text, displayed adjacent to the company name field: "Company name self-reported by engineer." This is not optional styling — it is the mechanism by which the card's two layers (engineer disclosure vs. system verification) remain legibly distinct to anyone viewing it, including the engineer's employer if they ever do.

8.4 What This Means in Practice

An engineer is free to name their employer on their own Proof Card, exactly as they are free to name an employer on a CV. What changes is that Proofboard's classification system is never the source of that name, never asserts it, and never couples it programmatically to the

percentage breakdown beneath it. The verified data stands on its own, independent of whatever the engineer chooses to disclose about who it was for.

9. Features Removed and Not Reintroduced

REMOVED

Post-Employment Log Parser / Wasm browser upload (originally Section 11, v1.2–v1.4) — removed in v1.5, remains removed. Engineers without current access now use the standard local CLI pipeline directly (Section 6.4) instead of a separate browser-based upload path. This is a cleaner and more consistent solution to the same underlying need.

REMOVED

Org Handshake (Phase 6, all versions through v1.6) — removed in this revision. See Section 6 for the replacement mechanism.

REMOVED

--skip-handshake flag and associated fallback documentation — removed. No longer applicable; there is no handshake.

10. NDA and Compliance Position

10.1 What the CLI Never Stores, Transmits, or Connects To

Category	Detail
Commit message text, file paths, code diffs, file content	Never stored, never transmitted, in any version.
Raw repository or organisation names	Never stored or transmitted in plaintext or as a static hash. Only per-engineer salted hashes (Section 4A).
Author email addresses	Stored and transmitted only as per-engineer salted hashes.
The repository's remote host (GitHub, GitLab, self-hosted, etc.)	NEVER CONNECTED TO. No live network call of any kind is made to the repository's infrastructure, by the engineer's credentials or otherwise. This is the headline change in v1.7.
Post-employment retained data of any kind	No special upload path exists or is needed. Standard local pipeline applies identically regardless of current employment status — see Section 6.4.

10.2 What the CLI Stores and Transmits

Category	Data
Cryptographic evidence	Salted SHA hash strings. Unix timestamps.
Numerical statistics	Additions, deletions, files changed per commit.
Classification output	Abstract technical category labels only — never business-domain labels.
Local fraud-detection scores	aiNoiseScore, commitSignatureVerified, signedCommitRatio — all computed locally.
Attestation	Salted orgHash and emailHash, cliVersion, dictionaryVersion.

11. Implementation Priorities

This Revision

Remove Phase 6 org handshake and all --skip-handshake handling. Implement local commit signature verification (Section 6.1). Extend aiNoiseScore with temporal entropy analysis (Section 6.2). Update proofboard status to reflect Locally Verified labelling instead of handshake-derived status. Update SHREDDER.md and README to describe the no-handshake architecture and local fraud-detection layer.

Web Application — Coordinate with Frontend Team

Decouple the Proof Card's organisation name field from any CLI-derived data source (Section 8). Add the 'Company name self-reported by engineer' label requirement. Ensure no autocomplete or suggestion logic on that field references the salted org hash or any data captured during proofboard sync.

Unchanged From v1.6

Salted hashing (Section 4A), DNS kill switch (Section 5), generic category vocabulary, no Wasm upload path, branch filter gate, trivial commit filter, binary distribution and code signing.

PROOFBOARD

CLI & Platform Integration Specification

Version 1.8 | June 2026

Implementation language: Go (CLI) · Next.js / TypeScript (Platform) | Open source: MIT (CLI) | Audience: CLI Engineering + Frontend team

> **LEGAL NOTE**
> This revision adds four things to v1.7: (1) the notification architecture is finalised at four notifications, replacing the prior three; (2) a platform integration section documents how the CLI's auth flow coordinates with the Next.js frontend's token storage, since the CLI and web app share the same authenticated user session; (3) the typography system used to render Proof Cards is documented, since font choice is treated as a product signal distinguishing verified data from human narrative; (4) a new legal section documents the copyright fact-versus-expression framing, Terms of Service self-certification requirement, and milestone content moderation policy. This document is an engineering specification, not legal advice. Proofboard should obtain review from a qualified employment / IP / data-privacy attorney before any feature in this document is used against a third party's private repository at scale.

Summary of changes from v1.7:

- Notification architecture finalised at FOUR notifications, not three: new project detection, added-to-profile confirmation, inbound opportunity, and monthly career summary. See Section 5A.
- The former 'Proof-of-Ship terminal echo' (fired on sync completion) is replaced by 'Added to profile' — fires only once the project is fully processed and live on the public profile, not on every merge.
- New Section 13: Platform Integration — documents the Next.js frontend's auth token storage pattern and how the CLI's emailHash bridge coordinates with it.
- New Section 14: Typography System — documents the three-font system used across the platform and specifically on the Proof Card.
- New Section 15: Copyright Position and Content Policy — documents the facts-versus-expression legal framing, Terms of Service self-certification, and milestone outcome statement content moderation.
- All architecture from v1.7 — no org handshake, local fraud detection, salted hashing, generic categories, engineer-authored company name, recency/consistency signals — remains unchanged and in effect.

5A. Notification Architecture

CHANGED IN v1.8

v1.8 finalises the notification list at FOUR, not three. The prior 'Proof-of-Ship terminal echo,' which fired immediately on sync completion, is replaced by 'Added to profile,' which fires only once the synced project has been fully processed server-side and is live on the engineer's public profile. This is a deliberate change: the engineer should be notified when there is something to actually look at, not when a background job finished. A fourth notification — Inbound Opportunity — is added, sourced from the B2B matching system rather than the CLI, and is included here because it completes the platform's full notification surface.

Exactly Four Notifications. Ever.

#	Notification	Trigger	Channel
1	New repo detected	CLI detects an unlinked Git repository	Terminal only
2	Added to profile	Project fully processed and live at proofboard.io/username — NOT on sync completion, NOT on every merge	Terminal, once, plus dashboard
3	Inbound opportunity	A company's job posting matches the engineer's verified work (Discoverability must be ON)	Dashboard + email
4	Monthly summary	Last Friday of each month	Email + one-time terminal line on next project open

5A.1 New Repo Detected

Unchanged from v1.7. Terminal-only prompt when the CLI detects a Git repository that has not been linked.

```
Proofboard — Git repository detected  Project: fintech-payments-api Last commit:
3 hours ago  Add this project to your career ledger?    y   Sync this project   n
Not this project   x   Never ask for this workspace
```

5A.2 Added to Profile

NEW IN v1.8

This replaces the 'Proof-of-Ship terminal echo' from all prior versions. The CLI still syncs silently in the background the moment the engineer approves — no notification fires at that point. The notification fires later, once, when the server has finished classification and the project actually appears on the public profile. This is a meaningfully different trigger point: the engineer is told when something changed that they can go look at, not when an internal processing step completed.

```
Proofboard: New project added to your profile. Click to view →
proofboard.io/divine-chuks
```

Property	Spec
Trigger	Server-side: project has completed classification (Phases 2-4) and is published to the public profile.
Frequency	Once per project, the first time it goes live. Subsequent syncs to the same project (new milestones, updated stats) do NOT re-fire this notification.
Channel	Terminal line on next CLI invocation after publish, and a dashboard notification badge.
What it explicitly does not do	Does not fire on sync completion. Does not fire per merge. Does not fire per milestone bundle — milestone highlights remain silent and held in-app for engineer review, unchanged from prior versions.

5A.3 Inbound Opportunity

> **NEW IN v1.8**
> New in v1.8. Sourced from the B2B matching system (verify.proofboard.io), not the CLI. Included here for completeness of the platform's full notification surface.

Fires when a recruiter's job posting matches the engineer's verified work profile. Requires the engineer's Discoverability toggle to be ON — if Discoverability is OFF, this notification never fires, full stop.

Property	Spec
Trigger	Match algorithm identifies overlap between a posted role and the engineer's verified category profile and Reputation Index.
Precondition	Discoverability toggle must be ON. Default state is OFF.
Channel	Dashboard Inbound section + email.
Company identity	Never disclosed to the engineer at this stage. Company type only (e.g. 'Series B fintech'). Full identity revealed only after the engineer opts in and completes screening.
Frequency cap	No hard cap specified in this revision — left as an open product decision. Recommend capping to avoid notification fatigue; see Section 5A.4.

5A.4 Monthly Career Summary

Unchanged from v1.7. One email per month, last Friday. Plus one quiet terminal line the first time the engineer opens any linked project after that month's summary generates. Fires once and clears — does not repeat on subsequent project opens that month.

```
Proofboard: Your March career summary is ready. proofboard.io/career-summary
```

5A.5 What Remains Silent — No Notification

Item	Behaviour
Milestone highlights	Generated automatically from contribution data, held in the application. Engineer reviews and approves at their own pace on the dashboard. No

Item	Behaviour
	notification is ever sent about pending milestones.
Every individual sync / merge	Only the first-publish event notifies (5A.2). Ongoing syncs to an already-published project are silent.
Quarterly snapshot	Exists as a dashboard feature, viewable anytime. No notification fires when it generates.
Weekly progress report	Does not exist. Removed in earlier revisions and not reinstated.

13. Platform Integration (NEW IN v1.8)

This section documents how the CLI's authentication flow coordinates with the Next.js web application's session management, since both surfaces authenticate the same engineer and must resolve to the same user account.

13.1 Frontend Stack

Layer	Technology
Framework	Next.js
Language	TypeScript
Styling	Tailwind CSS
Component library	shadcn/ui — used for modals, dialogs, and similar overlay components
Authentication providers	GitHub OAuth and GitLab OAuth only. No email/password. No other identity providers.

13.2 Web Session Token Storage

The web application stores the authenticated session client-side using two localStorage-backed stores. These are separate from, but must stay consistent with, the CLI's own credential storage at ~/.proofboard/credentials.json.

```
const TOKEN_KEY = "pb-token"; const USER_KEY = "pb-user";  export const
tokenStore = {   get: (): string | null => {     if (typeof window ===
"undefined") return null;     return localStorage.getItem(TOKEN_KEY);   },   set:
(token: string): void => {     if (typeof window === "undefined") return;
localStorage.setItem(TOKEN_KEY, token);   },   clear: (): void => {     if
(typeof window === "undefined") return;     localStorage.removeItem(TOKEN_KEY);
localStorage.removeItem(USER_KEY);   },   isAuthenticated: (): boolean =>
{     return !!tokenStore.get();   }, };  export const userStore = {   get: <T =
unknown>(): T | null => {     if (typeof window === "undefined") return null;
const raw = localStorage.getItem(USER_KEY);     if (!raw) return null;     try {
return JSON.parse(raw) as T;     } catch {       return null;     }   },   set:
(user: unknown): void => {     if (typeof window === "undefined") return;
localStorage.setItem(USER_KEY, JSON.stringify(user));   },   clear: (): void => {
if (typeof window === "undefined") return;     localStorage.removeItem(USER_KEY);
}, };
```

13.3 Why This Matters for the CLI Team

The web app's pb-token and the CLI's ~/.proofboard/credentials.json JWT are issued from the same auth backend but stored independently on different surfaces (browser localStorage vs. local filesystem). They are not automatically synchronised. The emailHash identity bridge (Section 7.2, unchanged from prior versions) is what ties a CLI session to the correct account when both surfaces are used by the same engineer — it does not synchronise tokens, it ensures both surfaces resolve to the same underlying user record.

Scenario	What Happens
Engineer signs up on the web app first, installs CLI later	Web app issues pb-token via GitHub/GitLab OAuth, stored via tokenStore. CLI's proofboard auth later issues its own JWT, and the emailHash bridge matches it to the same account.
Engineer installs CLI first, visits web app later	CLI issues its own JWT and perEngineerSalt on first proofboard auth. When the engineer later logs into the web app via GitHub/GitLab OAuth, the same emailHash matching applies in reverse.
Engineer logs out on the web app	tokenStore.clear() removes pb-token and pb-user from localStorage. This does NOT affect the CLI's separate credentials file — the CLI session remains authenticated independently.
Engineer revokes CLI access	proofboard unlink or credential deletion on the CLI side does not log the engineer out of the web app. The two sessions are independent by design.

Server-side, isAuthenticated() equivalents exist for both surfaces independently. There is no shared client-side session — each surface checks its own local credential store and validates against the same backend.

14. Typography System (NEW IN v1.8)

Font choice on the platform, and specifically on the Proof Card, is treated as a product signal — not a purely visual decision. The type system is designed to make the distinction between machine-verified data and human narrative visible at a glance, reinforcing the disclosure architecture in Section 8.

Font	Token	Used For
Open Sans	font-sans	Body copy — the default for all paragraph text across the platform.
DM Sans	font-display	Headings and display text — headlines, titles, subheadings.
Geist Mono	font-mono	Monospace labels — eyebrow tags, countdown labels, code-adjacent UI text, and machine-derived data on the Proof Card (commit counts, timestamps, hashes).

14.1 Why This Matters on the Proof Card

The Proof Card's disclosure architecture (Section 8) separates engineer-authored content from algorithm-derived data. Typography reinforces this separation visually, independent of the explicit self-reported label:

- Commit counts, PR counts, timestamps, category percentages — rendered in Geist Mono (font-mono), signalling machine-derived, verified data.
- The engineer's company name, role, and milestone outcome statements — rendered in the platform's serif treatment for narrative content (unchanged from prior card designs), signalling human-authored disclosure.
- Section headers and card titles — DM Sans (font-display).
- All other body copy, including the self-reported label text and verification descriptions — Open Sans (font-sans).

This means an engineer or recruiter can distinguish verified data from self-reported narrative by typography alone, before reading the explicit labels — the type system is doing communicative work, not just decorative work.

15. Copyright Position and Content Policy (NEW IN v1.8)

> **LEGAL NOTE**
> This section documents the primary legal framing for what Proofboard stores and displays, and introduces two product requirements — Terms of Service self-certification and milestone content moderation. This is the strongest available legal argument for the platform's data model. It is not a guarantee against disputes; see Section 16 (Legal Risk Register) for what remains open.

15.1 The Facts-Versus-Expression Framing

Under copyright law, an employer owns the expressive work — source code, code comments, file names, and architecture. Employers do not own historical facts: the number of commits made, the percentage breakdown of work by category, or the dates of a contribution. Proofboard is designed so that everything transmitted and displayed falls on the facts side of this line.

What Proofboard Never Touches	What Proofboard Stores and Displays
Source code, in any form	Numerical statistics — commit counts, additions, deletions
Code comments	Timestamps and date ranges
File names or paths	Abstract technical category labels and percentages
Architecture or system design detail	Generalised, non-identifying summaries derived from the above

This framing is the basis for the position Proofboard's legal counsel can take if a company raises a concern: the platform hosts a record of metadata and aggregated statistics, not code, trade secrets, or proprietary expression.

15.2 Terms of Service — Self-Certification Requirement

> **NEW IN v1.8**
> Product requirement, not yet drafted as final legal language — to be finalised with counsel. Documented here so the engineering and design teams build the corresponding UI acknowledgment at signup and at CLI first-run.

The Terms of Service must include a clause requiring the engineer to self-certify that they are authorised to generate a summary of their own contribution history. Proofboard relies on this self-certification and does not independently verify repository ownership or employment authorisation. This does not eliminate the underlying legal question addressed throughout this specification's revision history, but it establishes that responsibility for the underlying authorisation rests with the individual user, disclosed clearly at the point of use — not silently assumed.

Touchpoint	Requirement
Web signup	Checkbox acknowledgment referencing the Terms of Service self-certification clause, required before account creation completes.
CLI first-run (proofboard auth)	Terminal-printed summary of the same self-certification requirement, with a link to the full Terms of Service, before the first sync can run.

15.3 Milestone Outcome Statement — Content Policy

NEW IN v1.8

New product requirement. The engineer-authored outcome statement field (Section 8) is the one place on the platform where a user could paste content they should not — a confidential internal project name, a trademarked feature name, or other identifying detail.

Requirement	Spec
Inline warning	Displayed adjacent to the outcome statement textarea at all times, not just on first use: a short reminder not to include confidential names, internal project codenames, or proprietary details.
No automated content scanning at launch	Proofboard does not scan or block specific words or phrases in this field at launch — this avoids the platform positioning itself as a moderator or reviewer of content it does not otherwise process, and avoids false positives blocking legitimate generic descriptions.
Takedown mechanism	A reachable process (e.g. a dedicated email or form) for a company to request review or removal of a specific milestone statement they believe discloses something they should not have to see published. Response commitment should be fast — same-week at minimum — to demonstrate good faith if ever tested.
Terms of Service coverage	The self-certification clause in 15.2 extends to this field: the engineer is responsible for the content of their own outcome statements, same as for the underlying decision to generate a proof at all.

16. Legal Risk Register

> **LEGAL NOTE**
> This section is an engineering team's plain-language summary of known open risk for briefing counsel. It is not a legal opinion. Carried forward from v1.7 with two rows added reflecting this revision's additions.

Risk	Status	Residual Risk
Tool authenticates to or connects with the repository host's infrastructure	Eliminated (v1.7)	No outbound connection to the repository remote exists anywhere in the pipeline.
Organisation identity de-anonymisable via guess-and-check	Resolved (v1.6)	Per-engineer salting prevents cross-account correlation by company name.
Proof Card discloses employer-specific work allocation as a system-asserted claim	Resolved (v1.7)	Organisation name is engineer-authored, explicitly labelled self-reported, and now visually distinguished by typography (Section 14).
Fraudulent fabrication of commit history	Mitigated, not eliminated (v1.7)	Local signature verification and entropy analysis raise the cost of fabrication but do not make it impossible. Accepted trade-off.
Milestone outcome statements disclosing proprietary names or details	Mitigated (v1.8)	Inline warning and takedown mechanism reduce but do not eliminate the possibility of an engineer disclosing something they should not. No automated blocking at launch — see 15.3.
Platform liability for user self-certification claims later found false	Addressed via ToS (v1.8)	Self-certification shifts stated responsibility to the individual user but does not eliminate Proofboard's own exposure if it had reason to know a specific claim was false and acted on it anyway. This is a nuance for counsel, not resolved by the ToS clause alone.
Use of retained local data by an engineer no longer employed by the relevant company	Not resolved — unchanged open question	This is the same open legal question discussed throughout this specification's history: whether use of retained, abstracted, non-proprietary data derived from a former employer's repository requires that employer's consent. No engineering or product change in this document closes this question. It is fact-specific and requires legal review.

This document, together with the full revision history (v1.4 through v1.8), should be provided to qualified counsel alongside Proofboard's actual Terms of Service and Privacy Policy before the product is marketed or distributed for use against any employer's private repository at scale.

PROOFBOARD — v1.8

Build it. Open source it. Get it reviewed by counsel before scale.